Experiment No. 7
To implement value function approximation using linear regression and using neural network.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement value function approximation using **Linear Regression** and **Neural Networks**, and analyze their effectiveness in estimating value functions in reinforcement learning.

Objectives:

1. To understand the concept of **value function approximation** in reinforcement learning.
2. To implement **Linear Regression** as a function approximator for value estimation.
3. To implement a **Neural Network model** for approximating value functions..

Theory:

1. Value Function in Reinforcement Learning

In reinforcement learning, the **value function** estimates how good a state (or state-action pair) is in terms of expected future rewards.

- **State Value Function**:
  $V(s)$= Expected return starting from state s
- **Action Value Function**:
  $Q(s,a)$= Expected return for taking action $a$in state s

These functions guide the agent in selecting optimal actions.

2. Need for Function Approximation

In real-world problems:

- State space can be **very large or continuous**
- Storing values in a table (**tabular method**) becomes impractical

Solution:
Use function approximation to estimate value functions:

$$V(s) \approx \hat{V}(s,\theta)$$

Where:

- θ= model parameters
- $\hat{V}(s,\theta)$= approximated value

3. Linear Regression for Value Function Approximation

Concept

- Value function is approximated as a **linear combination of features**:

$$\hat{V}(s) = w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

or

$$\hat{V}(s) = \mathbf{w}^T\mathbf{x}$$

Where:

- **x**= feature vector representing state
- **w**= weight parameters

Working

1. Extract features from state
2. Predict value using linear model
3. Update weights using gradient descent

Advantages

- Simple and fast
- Easy to interpret

4. Neural Network for Value Function Approximation

Concept

Neural networks approximate value functions using **non-linear transformations**.

$$\hat{V}(s) = f(s, \theta)$$

Where:

- f is a neural network
- θ includes weights and biases

Working

1. Input: State representation
2. Hidden layers: Learn complex patterns
3. Output: Estimated value function

Advantages

- Can model **complex and non-linear relationships**
- Suitable for high-dimensional inputs

5. Learning Process

Both methods use:

- **Loss Function**: Mean Squared Error (MSE)

$$L = (V(s) - \hat{V}(s))^2$$

- **Optimization**:
  - Gradient Descent
  - Backpropagation (for neural networks)

6. Comparison of Linear Regression and Neural Networks

Feature	Linear Regression	Neural Network
Model Type	Linear	Non-linear
Complexity	Low	High
Training Time	Fast	Slower
Accuracy	Limited	High (for complex tasks)
Interpretability	High	Low

Implementation:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt

# Step 1: Create sample states (grid world)
```

```
states = np.array([[i, j] for i in range(5) for j in range(5)])

# True value function (example: distance from goal)
goal = np.array([4, 4])
values = -np.linalg.norm(states - goal, axis=1)

# -------------------------
# Linear Regression Model
# -------------------------
lin_model = LinearRegression()
lin_model.fit(states, values)

lin_predictions = lin_model.predict(states)

# -------------------------
# Neural Network Model
# -------------------------
nn_model = tf.keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(2,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(1)
])

nn_model.compile(optimizer='adam', loss='mse')

nn_model.fit(states, values, epochs=100, verbose=0)

nn_predictions = nn_model.predict(states).flatten()

# -------------------------
# Plot comparison
# -------------------------
```

```
plt.plot(values, label="True Values")
plt.plot(lin_predictions, label="Linear Regression")
plt.plot(nn_predictions, label="Neural Network")
plt.legend()
plt.title("Value Function Approximation")
plt.show()
```

Conclusion:

Value function approximation is essential in reinforcement learning when dealing with large or continuous state spaces.

- **Linear Regression** is simple, fast, and interpretable but limited to linear relationships.
- **Neural Networks** provide better accuracy and can model complex patterns but require more training and computational resources.

Thus, neural networks are more suitable for complex environments, while linear models are useful for simple problems.